

ILRM: Inverse Lag Recurrent Memory

Architecture, implementation, and preliminary experiments

Contact: rickeyernest1990@gmail.com

Publication date: 31 August 2026

Contents

Overview	2
1. Signal flow	2
2. Inverse-lag history	3
3. Why normalise the history?	4
4. Recurrent state and gate input	5
5. Gate 1: write gate	5
6. Gate 2: access gate	6
7. Candidate state	6
8. Final state update	6
9. Complete ILRM specification	7
10. Parameter dimensions	7
11. Why the inverse-lag path may help long-range learning	8
12. How ILRM can resist clutter	8
13. Efficient history calculation with FFT	8
Critical alignment rule	9
14. Minimal PyTorch-style reference implementation	10
15. Numerical implementation checks	11
16. Preliminary experiments: ILRM versus GRU	11
16.1 Long recall under continuously changing distractors	11
16.2 One-bit retention range	12
16.3 Gradient-horizon diagnostic	13
16.4 Hostile-clutter stress test	14
16.5 Causal gate intervention	14
16.6 Selective two-item memory	15

16.7 Corrected copy-memory screen	16
16.8 Relational fact-binding screen	16
16.9 Classic Adding Problem	16
17. What the experiments currently support	16
18. Reproducibility checklist	17
19. Short implementation-ready definition	17
20. Publication, contact and licensing	18
Licensing	18

Licensing warning - noncommercial use. This publication uses a split licence. The **documentation, explanatory text, figures, tables, and other copy-rightable presentation in this PDF** are licensed under [Creative Commons Attribution-NonCommercial 4.0 International \(CC BY-NC 4.0\)](#). **Source code and code listings implementing ILRM** are licensed separately under the [PolyForm Noncommercial License 1.0.0](#). Please review the applicable licence before copying, adapting, distributing, or reusing material from this publication.

Overview

ILRM (Inverse Lag Recurrent Memory) is an experimental recurrent neural-network cell designed to preserve useful information across long sequences while remaining compact and fully recurrent.

Its defining feature is a **normalised inverse-lag history** over previous input representations. Instead of requiring the recurrent hidden state to carry every useful detail from the distant past on its own, ILRM also gives the cell a direct weighted view of earlier inputs using the sequence

$$1, \quad \frac{1}{2}, \quad \frac{1}{3}, \quad \frac{1}{4}, \quad \frac{1}{5}, \dots$$

The most recent previous input receives the strongest weight, while older inputs fade progressively rather than falling outside a fixed context window.

At timestep t , ILRM works with three information sources:

$$x_t, \quad h_{t-1}, \quad P_t$$

where x_t is the current input representation, h_{t-1} is the previous recurrent hidden state, and P_t is the inverse-lag history of **previous input representations**.

Experimental status. ILRM is still an experimental architecture. The results later in this document are controlled screening experiments, not peer-reviewed evidence of general superiority over GRU or LSTM. The strongest evidence so far concerns long persistence under changing interference.

1. Signal flow

The safest way to read ILRM is as two learned gates built around a fixed inverse-lag history path.

First form the common gate input

$$z_t = [x_t; h_{t-1}; P_t].$$

Both gates are calculated from the same three information sources:

$$r_t = \sigma(W_r z_t + b_r), \quad w_t = \sigma(W_w z_t + b_w).$$

The access gate acts **inside candidate construction**:

$$r_t \odot h_{t-1} \longrightarrow \tilde{h}_t = \tanh(W_c[x_t; r_t \odot h_{t-1}; P_t] + b_c).$$

The write gate acts **after the candidate has been constructed**. It creates two final branches:

$$w_t \odot \tilde{h}_t$$

and

$$(1 - w_t) \odot h_{t-1}.$$

Those branches are added to produce the next recurrent state:

$$\boxed{h_t = (1 - w_t) \odot h_{t-1} + w_t \odot \tilde{h}_t}.$$

In compact flow form:

$$(x_t, h_{t-1}, P_t) \rightarrow (r_t, w_t) \rightarrow \tilde{h}_t \rightarrow \begin{cases} w_t \odot \tilde{h}_t \\ (1 - w_t) \odot h_{t-1} \end{cases} \rightarrow h_t.$$

This ordering matters: r_t controls access to the previous state while building the candidate; w_t controls how much of the completed candidate is written into persistent state.

2. Inverse-lag history

Let the current input representation be

$$x_t \in \mathbb{R}^{d_x}.$$

Define the harmonic number

$$\mathcal{H}_t = \sum_{k=1}^t \frac{1}{k}.$$

ILRM constructs its history vector as

$$P_t = \frac{\sum_{k=1}^t \frac{x_{t-k}}{k}}{\sum_{k=1}^t \frac{1}{k}},$$

with

$$P_0 = 0.$$

Equivalently,

$$P_t = \frac{1}{\mathcal{H}_t} \sum_{k=1}^t \frac{x_{t-k}}{k}.$$

A critical implementation rule is that P_t contains **previous inputs only**. The current input x_t must not appear inside P_t .

The first few history vectors are therefore

$$P_0 = 0,$$

$$P_1 = x_0,$$

$$P_2 = \frac{x_1 + \frac{1}{2}x_0}{1 + \frac{1}{2}},$$

and

$$P_3 = \frac{x_2 + \frac{1}{2}x_1 + \frac{1}{3}x_0}{1 + \frac{1}{2} + \frac{1}{3}}.$$

The underlying lag kernel is

$$K(k) = \frac{1}{k},$$

which is a power-law kernel with exponent 1.

3. Why normalise the history?

Without normalisation, the magnitude of

$$\sum_{k=1}^t \frac{x_{t-k}}{k}$$

can slowly increase as a sequence becomes longer. Dividing by $\mathcal{H}_t$ makes the coefficients sum to one:

$$\sum_{k=1}^t \frac{1/k}{\mathcal{H}_t} = 1.$$

Thus P_t behaves like a weighted average of previous inputs rather than an ever-growing accumulated sum.

The harmonic number grows approximately as

$$\mathcal{H}_t \approx \ln t + \gamma,$$

where γ is the Euler-Mascheroni constant. An input k positions in the past therefore has normalised contribution approximately

$$\frac{1}{k(\ln t + \gamma)}.$$

4. Recurrent state and gate input

Let

$$h_{t-1} \in \mathbb{R}^{d_h}$$

be the previous recurrent state. ILRM forms

$$z_t = [x_t; h_{t-1}; P_t].$$

Because both x_t and P_t have dimension d_x ,

$$z_t \in \mathbb{R}^{2d_x + d_h}.$$

This same combined representation is supplied to both learned gates.

5. Gate 1: write gate

The write/update gate is

$$w_t = \sigma(W_w z_t + b_w),$$

with

$$w_t \in (0, 1)^{d_h}.$$

The final state update can be rewritten as

$$h_t = h_{t-1} + w_t \odot (\tilde{h}_t - h_{t-1}).$$

If we define the candidate correction

$$\Delta_t = \tilde{h}_t - h_{t-1},$$

then the actual change applied to recurrent memory is

$$\Delta h_t = w_t \odot \Delta_t.$$

A small w_t can preserve the current state. A larger w_t permits more movement towards the candidate. Importantly, a large write-gate value does **not** automatically imply that the state is overwritten: if $\tilde{h}_t \approx h_{t-1}$, the effective state update is still small.

6. Gate 2: access gate

The second gate is

$$r_t = \sigma(W_r z_t + b_r).$$

It controls how much of the previous recurrent state participates in candidate construction:

$$r_t \odot h_{t-1}.$$

A useful interpretation is:

- **Access gate** r_t : controls how the existing recurrent memory participates in constructing the proposed new state.
- **Write gate** w_t : controls how much of that proposal is allowed to modify persistent memory.

7. Candidate state

The candidate state receives the current input, the access-gated previous state, and the inverse-lag history:

$$\tilde{h}_t = \tanh(W_c[x_t; r_t \odot h_{t-1}; P_t] + b_c).$$

Thus the candidate can combine current information, persistent learned state, and a direct long-tailed representation of earlier inputs.

8. Final state update

The recurrent state is updated by

$$h_t = (1 - w_t) \odot h_{t-1} + w_t \odot \tilde{h}_t.$$

Equivalently,

$$h_t = h_{t-1} + w_t \odot (\tilde{h}_t - h_{t-1}).$$

The second form gives an intuitive interpretation: the old state is the starting point, the candidate proposes a correction, and the write gate controls how much of that correction is accepted.

9. Complete ILRM specification

For an independent implementation, the entire basic cell is:

$$P_0 = 0,$$

$$P_t = \frac{\sum_{k=1}^t x_{t-k}/k}{\sum_{k=1}^t 1/k},$$

$$z_t = [x_t; h_{t-1}; P_t],$$

$$w_t = \sigma(W_w z_t + b_w),$$

$$r_t = \sigma(W_r z_t + b_r),$$

$$\tilde{h}_t = \tanh(W_c[x_t; r_t \odot h_{t-1}; P_t] + b_c),$$

$$h_t = (1 - w_t) \odot h_{t-1} + w_t \odot \tilde{h}_t.$$

For zero initialisation,

$$h_{-1} = 0.$$

The core ILRM definition contains no third gate, no separate LSTM-style cell state, no attention mechanism, no external key-value memory, and no learned decay exponent.

10. Parameter dimensions

For input width d_x and hidden width d_h ,

$$W_w, W_r, W_c \in \mathbb{R}^{d_h \times (2d_x + d_h)},$$

and

$$b_w, b_r, b_c \in \mathbb{R}^{d_h}.$$

With one bias vector per transformation, the recurrent cell contains

$$3d_h(2d_x + d_h) + 3d_h$$

trainable parameters.

11. Why the inverse-lag path may help long-range learning

In an ordinary recurrent network, an early input often affects a distant output mainly through a long chain of recurrent transitions. ILRM adds a second route because the old input also contributes directly to future P_t vectors.

For an input k steps old,

$$\frac{\partial P_t}{\partial x_{t-k}} = \frac{1}{k\mathcal{H}_t} I.$$

For the first input x_0 ,

$$\frac{\partial P_t}{\partial x_0} = \frac{1}{t\mathcal{H}_t} I.$$

Since $\mathcal{H}_t \sim \ln t$, this direct path decreases approximately as

$$\frac{1}{t \ln t}.$$

This is polynomial decay rather than an explicitly exponential path such as ρ^t .

This does **not** imply unlimited memory. Many previous inputs are still compressed into finite-dimensional representations, so interference, finite capacity, and optimisation can eventually become limiting factors.

12. How ILRM can resist clutter

ILRM does not need to deal with irrelevant inputs simply by forcing the write gate to zero. If a distractor produces

$$\tilde{h}_t \approx h_{t-1},$$

then

$$h_t = h_{t-1} + w_t \odot (\tilde{h}_t - h_{t-1})$$

naturally produces only a small state change even when w_t itself is not close to zero.

The intended behaviour is therefore closer to **active maintenance** than a hard memory lock: important events can produce a meaningful candidate change, while irrelevant clutter can produce a candidate that remains close to the persistent state.

13. Efficient history calculation with FFT

The numerator

$$N_t = \sum_{k=1}^t \frac{x_{t-k}}{k}$$

is a **linear causal convolution**. Define the fixed kernel

$$q = \left[0, 1, \frac{1}{2}, \frac{1}{3}, \frac{1}{4}, \dots\right].$$

Then, for the required causal samples,

$$N = X * q.$$

When FFTs are used, the convolution must be **zero-padded** so that the FFT computes a linear convolution rather than a circular one. For an input sequence of length T , choose an FFT length

$$L \geq 2T - 1$$

(often the next convenient fast FFT length), pad both X and q with zeros to length L , and compute

$$\widehat{N} = \text{IFFT}_L(\text{FFT}_L(X_{\text{pad}}) \odot \text{FFT}_L(q_{\text{pad}})).$$

Keep the first T causal samples:

$$N_t = \widehat{N}_t, \quad 0 \leq t < T.$$

Finally set

$$P_0 = 0,$$

and for $t \geq 1$,

$$P_t = \frac{N_t}{\mathcal{H}_t}.$$

A direct all-timestep implementation is approximately $O(T^2)$, whereas the zero-padded FFT convolution is approximately

$$O(T \log T).$$

The FFT does not redefine ILRM. It is a faster numerical evaluation of the same inverse-lag history, apart from ordinary floating-point round-off.

Critical alignment rule

The kernel must start with zero:

```
q[0] = 0
q[1] = 1
q[2] = 1/2
q[3] = 1/3
...
```

That leading zero ensures that the current input x_t is excluded from P_t . The zero-padding rule above is equally important: without sufficient zero-padding, an FFT implementation performs circular convolution and contaminates the history with wrap-around terms.

14. Minimal PyTorch-style reference implementation

The following code intentionally favours clarity over speed. The slow history function is useful as a reference against which an FFT implementation can be verified.

```
import torch
import torch.nn as nn

def inverse_lag_history_reference(x):
    """
    x: [batch, time, input_size]

    P[:, t] contains the normalised inverse-lag
    history of inputs strictly before timestep t.
    """
    B, T, D = x.shape
    P = torch.zeros_like(x)

    for t in range(1, T):
        numerator = torch.zeros_like(x[:, 0])
        denominator = 0.0

        for k in range(1, t + 1):
            numerator = numerator + x[:, t-k] / k
            denominator += 1.0 / k

        P[:, t] = numerator / denominator

    return P

class ILRMCCell(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()

        self.input_size = input_size
        self.hidden_size = hidden_size
        combined_size = 2 * input_size + hidden_size

        self.write_gate = nn.Linear(combined_size, hidden_size)
        self.access_gate = nn.Linear(combined_size, hidden_size)
        self.candidate = nn.Linear(combined_size, hidden_size)

    def forward(self, x_t, h_prev, P_t):
        gate_input = torch.cat([x_t, h_prev, P_t], dim=-1)

        w_t = torch.sigmoid(self.write_gate(gate_input))
```

```

r_t = torch.sigmoid(self.access_gate(gate_input))

candidate_input = torch.cat(
    [x_t, r_t * h_prev, P_t],
    dim=-1
)

h_candidate = torch.tanh(self.candidate(candidate_input))

h_t = (
    (1.0 - w_t) * h_prev
    + w_t * h_candidate
)

return h_t

```

15. Numerical implementation checks

The experimental implementation was checked against literal reference calculations before interpreting model results.

- Direct summation versus FFT history in float64: maximum absolute difference no larger than approximately 8.88×10^{-16} in the focused checks.
- During the final document recheck, the explicitly zero-padded FFT linear convolution was again compared with the direct history definition on random float64 sequences; the maximum absolute difference was 6.66×10^{-16} .
- Optimised/split ILRM cell versus a literal implementation of the equations: maximum absolute difference approximately 1.11×10^{-16} .
- The two algebraically equivalent update forms, $(1 - w_t) \odot h_{t-1} + w_t \odot \tilde{h}_t$ and $h_{t-1} + w_t \odot (\tilde{h}_t - h_{t-1})$, matched to float64 rounding precision in an independent recheck.
- A fresh float32 direct-versus-FFT recheck across the five trained long-memory seeds preserved every predicted class; the maximum logit difference was at most about 2.38×10^{-6} .

These checks are important because they indicate that the optimised implementation was evaluating the intended ILRM equations rather than an accidental variant.

16. Preliminary experiments: ILRM versus GRU

The model now called **ILRM** was named **Harmony** in the original experiment logs. Only the name has changed; the recurrent equations and reported experimental values are unchanged.

The tables below intentionally show **ILRM and the standard GRU only**. A separate GRU-plus-history model was also investigated during development, but it is omitted here to keep this document focused on the ILRM cell itself.

16.1 Long recall under continuously changing distractors

One of eight symbols was presented, followed by changing random distractor tokens, then a query asking for the original symbol. Chance accuracy was 12.5%.

The models were trained only on delays from **16 to 64 steps** and tested much farther without re-training. The focused confirmation used **five paired training seeds**, identical fixed evaluation examples, common optimiser settings, and parameter-matched models:

- GRU: 3,868 trainable parameters
- ILRM: 3,836 trainable parameters (0.83% fewer)

Distance	GRU	ILRM
64	12.8 +/- 1.2%	100.0 +/- 0.0%
128	12.7 +/- 1.5%	100.0 +/- 0.1%
256	12.0 +/- 2.0%	98.3 +/- 2.7%
512	12.5 +/- 1.8%	95.1 +/- 7.0%
1,024	12.1 +/- 1.5%	92.5 +/- 11.1%
2,048	11.9 +/- 1.3%	90.7 +/- 11.2%
4,096	12.5 +/- 1.3%	85.5 +/- 11.9%

At distance 1,024, the paired ILRM-minus-GRU difference was about **+80.4 percentage points**, with a preliminary paired 95% interval of approximately **+66.3 to +94.5 points**.

This is the strongest result obtained so far: under this synthetic long-memory task, ILRM retained task-relevant information through far longer changing input streams than the matched GRU.

Learning trajectory at an unseen distance of 1,024

Updates	GRU	ILRM
300	12.0%	17.6%
600	11.6%	36.0%
900	14.1%	87.6%
1,200	13.6%	90.7%
1,800	12.1%	92.2%

The matched GRU remained around chance during this budget, while ILRM learned a solution that extrapolated far beyond the training lengths.

Exploratory ultra-long 8-way recall

The same trained models were also probed beyond the well-sampled 4,096-step endpoint:

Distance	GRU	ILRM
8,192	9.7%	79.4%
16,384	16.9%	66.9%

These two points are exploratory: only 64 unique sequences were used at 8,192 and 32 at 16,384. They are useful for locating the beginning of the performance decline, but should not be treated as publication-grade endpoints.

16.2 One-bit retention range

A separate test stored one binary bit at the start of a sequence, inserted changing distractors, and queried the original bit at the end. Chance was 50%.

Training delays remained 16-64. The smaller parameter-matched models used:

- GRU: 1,252 parameters
- ILRM: 1,230 parameters (1.76% fewer)

Three independently trained paired seeds were tested.

Distance	GRU	ILRM	Unique fixed examples
64	50.2%	100%	512
256	50.5%	100%	512
1,024	50.0%	100%	512
4,096	50.3%	100%	512
8,192	51.0%	100%	64
16,384	51.0%	100%	32
32,768	43.8%	100%	16
65,536	54.2%	100%	8

The strongest well-sampled statement is therefore **100% accuracy at 4,096 steps on 512 fixed examples across all three trained ILRM models after training only up to delay 64**. The longer distances are exploratory lower bounds because the number of unique sequences was deliberately reduced to control CPU cost.

16.3 Gradient-horizon diagnostic

A separate diagnostic asked whether the long-memory behaviour was associated with a different gradient path, rather than merely better final predictions.

The already-trained 1,800-update checkpoints were used without further tuning. At each distance, GRU and ILRM received identical sequences. Gradients were computed in float64 to avoid confusing fp32 numerical underflow with vanishing gradients. The diagnostic scalar was the raw correct-class logit margin.

- **Token gradient:** $\|\partial m_T / \partial x_0\|_2$
- **Initial-state gradient:** $\|\partial m_T / \partial h_0\|_2$

Five-seed means after training were:

Distance	GRU token grad	ILRM token grad	GRU h_0 grad	ILRM h_0 grad
16	1.41e-05	6.04e-01	6.07e-05	1.01e+00
32	1.60e-08	5.09e-01	6.74e-08	8.28e-01
64	2.03e-14	8.00e-01	6.61e-14	1.74e+00
128	2.52e-26	6.56e-01	8.01e-26	1.40e+00
256	2.03e-49	5.99e-01	8.94e-49	1.28e+00
512	2.63e-95	6.43e-01	1.16e-94	1.13e+00
1,024	numerical underflow	4.93e-01	numerical underflow	9.61e-01
4,096	numerical underflow	1.08e+00	numerical underflow	1.85e+00

At random initialisation, the recurrent-state gradient still vanished strongly in ILRM, so the cell does not magically begin with a perfectly stable recurrent Jacobian. However, ILRM already had a direct token-to-history gradient path. At 1,024 steps, the mean initial token gradient was approximately 1.44×10^{-4} for ILRM versus approximately 2.80×10^{-145} for the GRU.

After training, ILRM had learned recurrent dynamics with order-one-scale task-relevant gradients through at least 4,096 steps in this diagnostic. The later accuracy decline at 8k-16k therefore cannot be explained solely by a classical vanished gradient at 4,096; interference, signal dilution, and finite capacity are plausible limiting factors.

16.4 Hostile-clutter stress test

Fresh examples at distance 1,024 deliberately made the distractors misleading. These were five-seed evaluations of previously trained parameter-matched models. Chance was 12.5%.

Distractor pattern	GRU	ILRM
Random changing distractors	12.7%	92.4%
Wrong symbol immediately before query	8.0%	93.2%
Final 25% all the same wrong symbol	5.5%	81.2%
Every distractor the same wrong symbol	4.9%	78.1%

The result argues against a simple strategy such as following the most recent or most frequent symbol.

16.5 Causal gate intervention

To test whether the two learned ILRM gates actually contributed to the clutter solution, their values were overridden during the distractor period at distance 1,024.

ILRM condition	Accuracy
Normal learned gates	92.6%
Force write gate $w_t = 1$	36.4%
Force write gate $w_t = 0$	36.7%
Force access gate $r_t = 1$	63.8%
Force access gate $r_t = 0$	12.1%

This is useful causal evidence. ILRM was not succeeding simply by permanently closing the write gate: forcing it either fully open or fully closed damaged performance. Disabling the access gate collapsed performance to approximately chance.

What the gates were doing during clutter

At distance 1,024, the average hidden-state update RMS across five seeds was:

Position	Mean w_t	Mean r_t	Hidden-state update RMS
Stored item	0.606	0.649	0.214
First distractor	0.641	0.732	0.413
Settled distractors	0.584	0.675	0.066
Mid distractor	0.583	0.673	0.063
Late distractor	0.581	0.670	0.067
Query	0.556	0.613	0.135

The write gate did not simply collapse towards zero. Instead, once the sequence settled, distractors produced much smaller actual state movements. This matches the update equation: clutter

can be suppressed because the candidate remains near the persistent state, not only because w_t closes.

16.6 Selective two-item memory

A more demanding experiment tested whether ILRM could preserve an early relevant value while accepting a later relevant value instead of simply locking its state after the first item.

Each sequence contained:

A_MARK, A_VALUE, clutter..., B_MARK, B_VALUE, clutter..., QUERY_A or QUERY_B

Values and clutter used eight symbols, so chance was 12.5%. The five-seed parameter-matched models used 3,904 GRU parameters and 3,872 ILRM parameters (0.82% fewer).

Query A: preserve the early item after B arrives

Distance	GRU	ILRM
64	12.9%	98.9%
256	13.1%	94.2%
1,024	12.4%	73.0%

Query B: accept the later relevant item

Distance	GRU	ILRM
64	13.0%	71.3%
256	12.5%	67.5%
1,024	12.7%	57.0%

The second-slot result had substantial seed-to-seed variability, so this is evidence of selective capability rather than proof of robust arbitrary multi-item memory.

A particularly informative intervention blocked the write gate only when B arrived:

- Normal: Query A = 99.3%, Query B = 71.4%
- Block B write: Query A = 97.0%, Query B = 12.0%

The B memory dropped to chance while A remained almost intact. This supports the interpretation that the cell can admit a later relevant item without inherently overwriting the earlier one.

Relevant events also produced much larger hidden-state movements than settled clutter at distance 1,024:

- A value: 0.706 RMS
- B marker: 0.851 RMS
- B value: 0.804 RMS
- first clutter segment: 0.111 RMS
- second clutter segment: 0.121 RMS

16.7 Corrected copy-memory screen

A corrected copy-memory experiment trained the loss on the actual recall segment so that predicting blank tokens could not dominate the objective. At distance 1,000, after 1,800 updates, the three-seed sealed-test token recall was:

- GRU: **15.9** +/- **4.1%**
- ILRM: **23.1** +/- **2.0%**

The models were closely parameter matched (GRU 3,909; ILRM 3,873, or 0.92% fewer parameters for ILRM).

No model achieved exact 10-token sequence recall in this small-compute screen, so this is evidence of improved **partial token recall**, not a solved copy-memory benchmark.

16.8 Relational fact-binding screen

A small synthetic fact-binding benchmark used fresh assignments such as person-colour-object relationships and then queried the correct binding. After longer training, the early apparent ILRM advantage disappeared.

On the three-seed sealed 4-fact endpoint:

- GRU: **25.3** +/- **0.9%**
- ILRM: **23.5** +/- **1.2%**

The parameter counts were GRU 5,677 and ILRM 5,642 (0.62% fewer for ILRM).

This benchmark therefore **does not currently support an ILRM advantage**. It is included because negative and neutral results are important when judging an experimental architecture.

16.9 Classic Adding Problem

The Adding Problem provides another useful counterexample to the long-memory result. After 1,800 updates, both models solved the task, but the GRU had the slightly lower sealed-test MSE:

- GRU: **0.000362** +/- **0.000111**
- ILRM: **0.000409** +/- **0.000066**

Lower is better. The models were again closely matched in parameter count (GRU 3,655; ILRM 3,616, or 1.07% fewer for ILRM).

This result does **not** support a general claim that ILRM is better than GRU. The strongest ILRM evidence is specifically in long persistence under changing interference.

17. What the experiments currently support

Taken together, the screening experiments support a narrower and more defensible picture than “ILRM beats GRU”.

Evidence currently favouring ILRM:

- much stronger retention under continuously changing distractors;
- large train-length extrapolation beyond the 16-64 training window;
- a direct inverse-lag gradient path that does not decay exponentially by construction;
- learned long-range recurrent dynamics in the trained diagnostic;
- resistance to recent and repeated wrong-symbol clutter;

- causal evidence that both learned gates participate in clutter control;
- evidence that a later relevant value can be written without automatically destroying an earlier one;
- better partial token recall on the corrected small copy-memory screen.

Results that do not favour ILRM:

- the classic Adding Problem slightly favoured GRU at the final endpoint;
- the longer-trained relational fact-binding screen did not show an ILRM advantage;
- selective second-slot memory still had noticeable seed-to-seed variability;
- the 8k-65k extreme-distance probes used progressively smaller fixed test sets and should be treated as exploratory.

These distinctions matter. The current evidence suggests that ILRM’s most promising property is **long persistence under interference**, not universal superiority across recurrent-network tasks.

18. Reproducibility checklist

An independent implementation should preserve the following points:

1. P_t is computed from **previous input vectors**, never from previous hidden states.
2. The history kernel is exactly $1/k$ and is normalised by the corresponding harmonic number.
3. $P_0 = 0$.
4. Both learned gates receive $[x_t; h_{t-1}; P_t]$.
5. Only the access-gated hidden state $r_t \odot h_{t-1}$ enters the candidate.
6. The final update is exactly

$$h_t = (1 - w_t) \odot h_{t-1} + w_t \odot \tilde{h}_t.$$

7. The basic cell has no third gate, no separate cell state, no attention, and no learned decay exponent.
8. If FFT is used, the convolution kernel begins with zero so that the current input is excluded from P_t , and both sequences are zero-padded to a length of at least $2T - 1$ so the FFT computes linear rather than circular convolution.
9. A new implementation should first be checked against a direct float64 history calculation and a literal cell implementation before benchmarking.

19. Short implementation-ready definition

ILRM (Inverse Lag Recurrent Memory) is a two-gate recurrent cell augmented with a normalised inverse-lag history of previous input representations. At timestep t ,

$$P_t = \frac{\sum_{k=1}^t x_{t-k}/k}{\sum_{k=1}^t 1/k}, \quad P_0 = 0.$$

Form

$$z_t = [x_t; h_{t-1}; P_t].$$

Compute

$$w_t = \sigma(W_w z_t + b_w)$$

and

$$r_t = \sigma(W_r z_t + b_r).$$

Construct

$$\tilde{h}_t = \tanh(W_c[x_t; r_t \odot h_{t-1}; P_t] + b_c),$$

then update

$$h_t = (1 - w_t) \odot h_{t-1} + w_t \odot \tilde{h}_t.$$

The single most important rule is:

ILRM’s inverse-lag history is built from previous input representations, not hidden states.

20. Publication, contact and licensing

Publication date: 31 August 2026

Contact: rickeyernest1990@gmail.com

Licensing

Licensing warning - noncommercial use. Different parts of this publication are licensed under different noncommercial licences. Use the licence that applies to the material you are reusing.

Documentation, explanatory text, figures and tables

The copyrightable documentation and presentation in this PDF are licensed under the [Creative Commons Attribution-NonCommercial 4.0 International \(CC BY-NC 4.0\)](#). This permits sharing and adaptation subject to the licence terms, including attribution and the noncommercial-use restriction.

Source code and code listings

Source code implementing **ILRM**, including implementation code and source-code listings reproduced in this document, is licensed separately under the [PolyForm Noncommercial License 1.0.0](#).

Commercial use

Neither of the licences above grants unrestricted commercial-use rights. For commercial licensing enquiries, contact rickeyernest1990@gmail.com.

Scope note

These licence notices apply to the copyrightable expression, documentation, figures, presentation, and software supplied with this work to the extent protected by applicable law. They are not intended to restrict mathematical facts, formulas, abstract ideas, or other material that is not subject to copyright protection. The licence notices do not themselves establish or imply any claim about patentability, novelty, or ownership of ideas that may be governed by separate law or prior art.